

Курс: Углубленное машинное обучение и искусственный интеллект в Python

Практическое задание: Рекуррентные нейронные сети (RNN/LSTM) и классификация текста

Цель работы: Научиться строить и обучать рекуррентные нейронные сети (RNN/LSTM) для обработки текстовых последовательностей, использовать готовые токенизаторы из экосистемы Hugging Face, а также проводить автоматический подбор архитектуры и гиперпараметров модели с помощью Optuna.

Шаг 1. Поиск и выбор текстового датасета на Hugging Face

Выберите любой открытый датасет на платформе Hugging Face, предназначенный для задачи классификации текста (не регрессия).

Примеры подходящих датасетов:

Датасет	Описание	Количество классов
tweet_eval (subset emotion)	Определение эмоций в твитах	6 классов
imdb	Анализ тональности отзывов к фильмам	2 класса (positive / negative)
rotten_tomatoes	Рецензии на фильмы и их тональность	2 класса (pos / neg)
dair-ai/emotion	Классификация эмоциональной окраски текста	6 классов
stanfordnlp/sst2	Анализ тональности предложений (SST)	2 класса

Требования к датасету:

- Объем выборки: не менее 5000 примеров.
- Структура: наличие текстового поля и метки класса (целевой переменной).
- Количество классов: от 2 до 10.

Шаг 2. Загрузка данных и токенизатора

Загрузите выбранный датасет и предобученный токенизатор. **Важно:** в рамках этого задания мы используем токенизатор «как есть» и не обучаем собственные слои векторизации вроде TextVectorization.

```
from datasets import load_dataset
from transformers import AutoTokenizer

# Загружаем датасет
dataset = load_dataset("tweet_eval", "emotion")

# Берем готовый токенизатор с HuggingFace (не обучаем свой!)
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")

# Проверяем структуру полученных данных
print(dataset)
print(dataset["train"][0])  # Ожидаемый вывод: {'text': '...',
                             'label': 0}
```

Шаг 3. Подготовка и векторизация данных

Создайте функцию токенизации с фиксированной максимальной длиной последовательности, примените её ко всем сплитам данных и подготовьте массивы для передачи в Keras.

```
import numpy as np

def tokenize_function(examples):
    # Токенизируем текст с заполнением (padding) и усечением (truncation)
    return tokenizer(
        examples['text'],
        truncation=True,
        padding='max_length',
        max_length=100  # Максимальная длина последовательности
        токенов
```

```

    )

# Применяем токенизатор ко всем частям датасета (train / validation
/ test)
tokenized_dataset = dataset.map(tokenize_function, batched=True)

# Функция для преобразования в формат numpy-массивов для Keras
def to_keras_format(dataset_split):
    X = np.array(dataset_split['input_ids']) # Индексы токенов
    y = np.array(dataset_split['label'])     # Метки классов
    return X, y

X_train, y_train = to_keras_format(tokenized_dataset['train'])
X_val, y_val = to_keras_format(tokenized_dataset['validation'])
X_test, y_test = to_keras_format(tokenized_dataset['test'])

print(f"Train shape: {X_train.shape}")      # Ожидается: (n_samples,
100)
print(f"Vocabulary size: {tokenizer.vocab_size}") # Примерно ~30000

```

Шаг 4. Построение базовой модели (Baseline LSTM)

Создайте и обучите базовую конфигурацию рекуррентной нейронной сети со следующим пайплайном слоев:

```

import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Embedding(
        input_dim=tokenizer.vocab_size,
        output_dim=128,
        mask_zero=True # Игнорируем pad-токены при вычислениях
    ),
    tf.keras.layers.LSTM(64, return_sequences=False),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(num_classes, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

```

```
# Обучение модели
history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=10,
    batch_size=64,
    verbose=1
)

baseline_accuracy = max(history.history['val_accuracy'])
print(f"Baseline val accuracy: {baseline_accuracy:.4f}")
```

Шаг 5. Оптимизация гиперпараметров через Optuna

Настройте процесс автоматического поиска оптимальной архитектуры сети и параметров обучения. Вам необходимо оптимизировать следующие параметры в заданных диапазонах:

Параметр	Диапазон поиска	Описание / Влияние
embedding_dim	[64, 128, 256]	Размерность плотных векторов эмбеддингов слов.
lstm_units	[32, 64, 128, 256]	Количество скрытых состояний (нейронов) в слое LSTM.
n_lstm_layers	[1, 2]	Глубина рекуррентной части модели (количество последовательных LSTM слоев). При n=2 не забудьте указать return_sequences=True на первом слое.
dropout	[0.2, 0.3, 0.4, 0.5]	Коэффициент Dropout после рекуррентных слоев для регуляризации.
recurrent_dropout	[0.0, 0.2, 0.3]	Коэффициент Dropout для внутренних связей ячеек LSTM.
learning_rate	[1e-4, 3e-4, 1e-3]	Шаг оптимизатора Adam.
batch_size	[32, 64, 128]	Размер пакета данных при обучении.
max_length	[50, 100, 150]	Размер контекстного окна (длина токенизированной строки). Обратите

Параметр	Диапазон поиска	Описание / Влияние
		внимание: изменение этого параметра требует повторной подготовки массивов данных внутри trial-функции.

Шаг 6. Обучение финальной модели

По результатам оптимизации выделите наилучшую конфигурацию гиперпараметров. Инициализируйте финальную модель и обучите её на полном объеме данных (train + validation), чтобы добиться максимального качества генерализации.

Шаг 7. Сравнение результатов

Заполните сводную таблицу по итогам экспериментов и сделайте краткий аналитический вывод.

Модель	Val Accuracy	Test Accuracy	Параметры модели	Время обучения
Baseline (LSTM 64)	?	?	Стандартные (из Шага 4)	-
Optuna best	?	?	Оптимизированные Optuna	-